

UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación Paralela y Distribuida
Semestre 2, 2026

ENTREGA 2 - PROYECTO #1

Screensaver de fuegos artificiales con OpenGL

Integrantes

Adrián González - 23152

Fernando Mendoza - 19644

Hansel López - 19026

Guatemala, agosto de 2026

Índice

1. Investigación y selección de herramienta gráfica
 - 1.1 Herramienta seleccionada: OpenGL
 - 1.2 Justificación de la selección
 - 1.3 Requisitos básicos para instalación y uso con C/C++
 - 1.4 Ejemplo de referencia
 2. Prueba de concepto del screensaver
 - 2.1 Objetivo
 - 2.2 Relación con la propuesta de la Entrega 1
 - 2.3 Creación de la ventana gráfica
 - 2.4 Elemento visual implementado
 - 2.5 Movimiento y física
 - 2.6 Parámetro N
 - 2.7 Colores pseudoaleatorios
 - 2.8 Frames por segundo
 - 2.9 Preparación para la paralelización
 3. Resultado de la prueba de concepto
- Referencias

1. Investigación y selección de herramienta gráfica

1.1 Herramienta seleccionada: OpenGL

Para el desarrollo del screensaver se seleccionó OpenGL (Open Graphics Library) como herramienta principal de renderizado. OpenGL es una API gráfica multiplataforma orientada a la generación de gráficos 2D y 3D y puede aprovechar aceleración por hardware.

Se utilizará GLFW como biblioteca auxiliar para crear la ventana, establecer el contexto de OpenGL y procesar eventos básicos. También se utiliza GLEW (OpenGL Extension Wrangler Library) para cargar las funciones de OpenGL moderno de forma portable en el entorno elegido.

La configuración seleccionada para el proyecto es la siguiente:

- Lenguaje: C++17.
- Renderizado: OpenGL.
- Ventana y contexto gráfico: GLFW.
- Carga de funciones OpenGL: GLEW.
- Versión objetivo: OpenGL 3.3 Core Profile.

1.2 Justificación de la selección

La propuesta de la Entrega 1 define un screensaver que simula fuegos artificiales. Cada explosión genera N partículas con posición, velocidad, color y tiempo de vida; las partículas se dispersan, reciben el efecto de la gravedad y se desvanecen. Esta estructura se adapta bien al renderizado mediante OpenGL.

OpenGL resulta apropiado porque permite separar con claridad el cálculo de la simulación en CPU del renderizado gráfico. La CPU puede actualizar posición, velocidad, gravedad y vida de las partículas, mientras OpenGL dibuja el estado resultante. Esa separación favorece la etapa posterior del proyecto, cuando la actualización independiente de las partículas sea paralelizada con OpenMP.

Adicionalmente, OpenGL es multiplataforma, tiene amplia documentación y permite manejar eficientemente una cantidad elevada de elementos gráficos, lo cual es conveniente para evaluar el comportamiento de la aplicación con distintos valores de N.

1.3 Requisitos básicos para instalación y uso con C/C++

Se requieren los siguientes componentes:

- Compilador compatible con C++17.
- CMake 3.16 o superior.
- OpenGL.
- GLFW 3.3 o superior.
- GLEW.

En Ubuntu, Debian o WSL2 se pueden instalar las dependencias con:

```
sudo apt update  
sudo apt install build-essential cmake libglfw3-dev libglew-dev libgl1-mesa-dev
```

Para compilar el proyecto:

```
cmake -S . -B build  
cmake --build build
```

Para ejecutarlo con 500 partículas:

```
./build/screensaver 500
```

1.4 Ejemplo de referencia

Un ejemplo de referencia para una aplicación OpenGL es la creación de una ventana con GLFW y la ejecución de un ciclo de renderizado. El flujo general consiste en inicializar GLFW, crear una ventana y contexto, cargar las funciones de OpenGL, ejecutar el ciclo de dibujo, intercambiar buffers y procesar eventos.

Este patrón aparece en la documentación de GLFW y en el tutorial “Hello Window” de LearnOpenGL, y constituye la base técnica utilizada por la prueba de concepto.

2. Prueba de concepto del screensaver

2.1 Objetivo

La prueba de concepto busca comprobar que el equipo puede:

- Crear una ventana gráfica con GLFW y OpenGL.
- Inicializar el pipeline de renderizado.
- Generar dinámicamente N partículas.
- Mostrar partículas de distintos colores.
- Animar las partículas en tiempo real.
- Aplicar gravedad.
- Mostrar los FPS actuales y el valor de N.
- Mantener la estructura preparada para paralelización posterior.

Esta implementación corresponde todavía a una versión secuencial inicial; OpenMP no forma parte del alcance de esta Entrega 2.

2.2 Relación con la propuesta de la Entrega 1

La propuesta original describe un espectáculo de fuegos artificiales. Cada explosión genera N partículas con una posición y velocidad iniciales. En cada frame se actualiza la velocidad vertical aplicando gravedad, luego se modifica la posición y finalmente se reduce la vida de la partícula hasta que desaparece.

El movimiento implementado sigue una aproximación discreta de las siguientes ecuaciones:

```
vy = vy + g * dt
x  = x + vx * dt
y  = y + vy * dt
```

Cuando todas las partículas de una explosión desaparecen, se genera automáticamente una nueva explosión en una posición pseudoaleatoria.

2.3 Creación de la ventana gráfica

La prueba de concepto crea una ventana de 800 x 600 píxeles, superior al mínimo de 640 x 480 establecido en las instrucciones generales del proyecto. La aplicación utiliza OpenGL 3.3 Core Profile y permanece activa hasta que el usuario presiona ESC o cierra la ventana.

2.4 Elemento visual implementado

El elemento implementado es la explosión de partículas del fuego artificial. Cada explosión genera exactamente N partículas. Para cada partícula se generan pseudoaleatoriamente:

- dirección inicial;
- velocidad inicial;
- componentes rojo, verde y azul;
- tiempo de vida.

Las partículas se dibujan como puntos circulares sobre un fondo oscuro. El fragment shader utiliza `gl_PointCoord` para descartar los fragmentos fuera del radio, produciendo una partícula circular en lugar de un cuadrado.

2.5 Movimiento y física

Durante cada frame se ejecutan los siguientes pasos:

1. Se aplica gravedad a la velocidad vertical.
2. Se actualiza la posición horizontal.
3. Se actualiza la posición vertical.
4. Se reduce el tiempo de vida.
5. Se reduce la opacidad visual.
6. Las partículas apagadas dejan de renderizarse.

El resultado produce trayectorias similares a parábolas, cumpliendo desde esta etapa con el componente de física solicitado para el screensaver.

2.6 Parámetro N

El parámetro N representa la cantidad de partículas generadas por cada explosión y puede proporcionarse desde línea de comandos:

```
./build/screensaver 500
```

La aplicación valida defensivamente el argumento. El rango definido para la prueba de concepto es:

```
1 <= N <= 200000
```

Si no se proporciona un valor, se utilizan 500 partículas.

2.7 Colores pseudoaleatorios

Cada partícula recibe componentes RGB pseudoaleatorios. Se evita utilizar colores demasiado oscuros para que las partículas sean visibles sobre el fondo, manteniendo el requisito de desplegar varios colores.

2.8 Frames por segundo

La aplicación calcula los FPS de forma periódica y los muestra junto con el valor de N en el título de la ventana, por ejemplo:

```
OpenGL Fireworks | N = 500 | FPS = 60.0
```

Esto permite comenzar a observar el comportamiento del programa cuando se modifica el número de partículas y deja preparada la instrumentación necesaria para pruebas de rendimiento posteriores.

2.9 Preparación para la paralelización

La Entrega 2 mantiene el programa secuencial. La actualización de partículas se encuentra aislada dentro de `updateParticles()`. Cada partícula modifica únicamente su propio estado, por lo que este ciclo constituye un candidato natural para utilizar posteriormente `#pragma omp parallel for`.

El renderizado con OpenGL permanece fuera de esa futura sección paralela, manteniendo separado el acceso al contexto gráfico de los cálculos físicos.

3. Resultado de la prueba de concepto

La prueba de concepto cubre los objetivos de esta entrega:

- Selección definitiva de OpenGL.
- Descripción y justificación de la herramienta.
- Requisitos básicos de instalación y uso.
- Referencia de una aplicación gráfica.
- Ventana OpenGL funcional de 800 x 600.
- Visualización de N partículas.
- Colores pseudoaleatorios.
- Movimiento y gravedad.
- Desvanecimiento de partículas.
- Nuevas explosiones durante la ejecución.
- FPS visibles.
- Estructura preparada para incorporar OpenMP posteriormente.

No se incorpora aún la paralelización, el cálculo de speedup o eficiencia, ni la implementación completa de todas las características previstas para la entrega final del proyecto.

Referencias

1. Universidad del Valle de Guatemala. Proyecto #1 - Computación Paralela y Distribuida, Semestre 2, 2026.
2. González, A.; Mendoza, F.; López, H. Propuesta - Proyecto #1: Screensaver paralelo con OpenMP, 2026.
3. Khronos Group. OpenGL. <https://www.khronos.org/opengl/>
4. GLFW Project. GLFW Documentation. <https://www.glfw.org/documentation.html>
5. de Vries, J. LearnOpenGL - Hello Window. <https://learnopengl.com/Getting-started/Hello-Window>